

Compiler Register Allocation

01
10



DA Programming Project II

Group T9G2 / Spring 2026

Alexandre Teixeira

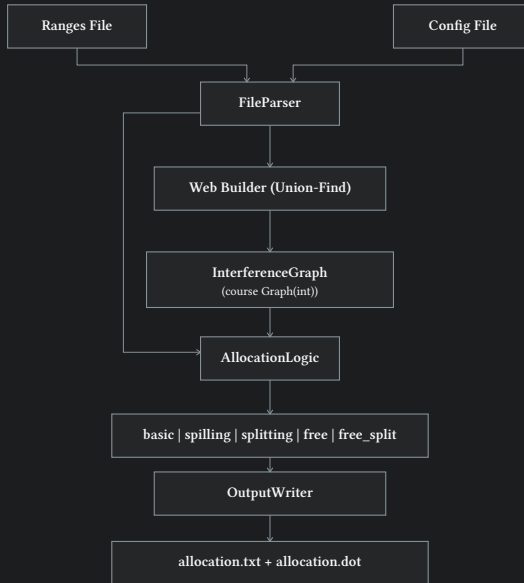
Carlos Diogo

Rodrigo Dias

`make test`

`make docs`

`./register_alloc -b ...`



Input Format

Live ranges file

```
sum: 7+,8,9,10-  
i: 1+,2,3,4,5,6-  
i: 9+,10,11,12-
```

- var: comma-separated line numbers
- + = definition, - = last use
- same variable → multiple ranges
- # starts a comment

Register config file

```
registers: 2  
algorithm: spilling, 1
```

- registers: number of physical regs
- algorithm: basic | spilling,N | splitting,N | free | free_split
- missing algorithm defaults to basic

Output Format

Allocation file

```
webs: 3  
web0: 1+,2,3,4,5,6-  
web1: 9+,10,11,12,14-,20+  
web2: 7+,8,9,10-
```

```
registers: 2  
r0: web0  
r0: web2  
r1: web1
```

Sections

- **Web list:** webN: sorted points
- **Assignment:** rN or M per web
- **Metadata:** spills/splits record

DOT output

- Same register = same fill color
- Memory webs → gray boxes
- Split webs → orange border
- Written alongside allocation

Basic Algorithm

Core coloring loop

- Build interference graph from webs
- Simplify: remove node with degree $< K$, push to stack
- Spill candidate: select high-degree node when no low-degree node exists
- Assign: pop stack, assign available register or mark for spill
- Rewrite program and repeat until colored or bounded spilling applied

Notes: K = number of physical registers. Spill heuristic: minimize cost/degree ratio.

Spilling and Splitting

01
10

Bounded spilling

Colored Register Allocation
2 register(s), 1 memory web(s)

spills: web0

Legend

orange border = split-derived

M = memory

same fill = same register

web0
a
M
1+,2,3-

web1
b
r1
1+,2,3-

web2
c
r0
1+,2,3-

Bounded splitting

Colored Register Allocation
2 register(s), 0 memory web(s)

splits: web0 -> web0,web3

Legend

orange border = split-derived

M = memory

same fill = same register

web0
a
r0
1+,2

web1
b
r1
1+,2,3,4-

web2
c
r0
3+,4,5,6-

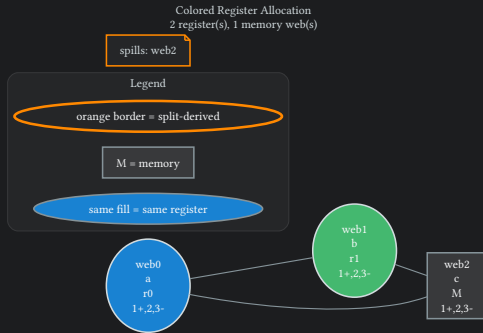
web3
a
r1
5,6-

Splitting preserves original markers. 1+,2,5,6- splits into 1+,2 and 5,6- - no fabricated 2- or 5+. Metadata: spills: N / splits: N.

Custom Free Algorithms

Hybrid strategy

- Edgeless graphs: 1 register
- Complete graphs: K colored, rest spilled
- Bipartite graphs: BFS 2-coloring
- General: DSatur greedy coloring
- Small/medium: branch-and-bound refines spills
- free_split: split only after spills remain



free never changes the graph. free_split first runs free, then rebuilds/recolors candidate splits and keeps only verified improvements.

Demo and Testing

Inputs for the demo

- complex_allocation.txt + free4.txt: normal free with spills
- splitting_showcase.txt + splitting2.txt: split reduces K from 3 to 2
- spilling5.txt: bounded spilling in a dense graph

```
./register_alloc -b  
  inputs/advanced/ranges/  
complex_allocation.txt  
  inputs/advanced/registers/free4.txt  
alloc.txt
```

make test - 85 unit assertions + 9 integration cases. DOT graphs written alongside allocation output.

Complexities

Input & graph building

Parse: $O(LP)$

Build webs: $O(VR^2P \log P)$

Interference: $O(W^2P + EW)$

Coloring algorithms

Basic: $O(W(W^2 + E))$

Spilling: $O((S + 1)W(W^2 + E))$

Splitting: $O(SQ(W^2P + EW + W(W^2 + E)))$

Free: $O(W^2 + E \log W)$ + guarded exact

Free split: $F + SQ(W^2P + F)$

Variables: W=webs, E=edges, P=points, R=ranges/var, S=recovery bound, Q=split candidates, L=live ranges.